Atelier 08 Spring Security : JDBC, JPA et UserDetails

Objectifs :

1. L'authentification en utilisant JDBC/Mysql,
2. Personnaliser l'interface Login et Logout,
3. L'authentification en utilisant JPA/Mysql

L'authentification en utilisant JDBC/Mysql

1. Créer dans la base de données spring_db les tables USER, ROLE, et USER_ROLE :

```
CREATE TABLE `user` (
  `user_id` bigint(20) NOT NULL,
  `enabled` bit(1) DEFAULT NULL,
  `password` varchar(255) DEFAULT NULL,
  `username` varchar(255) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE `role` (
  `role_id` bigint(20) NOT NULL,
  `role` varchar(255) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE `user_role` (
  `user_id` bigint(20) NOT NULL,
  `role_id` bigint(20) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

ALTER TABLE `role`   ADD PRIMARY KEY (`role_id`);

ALTER TABLE `user`   ADD PRIMARY KEY (`user_id`);

ALTER TABLE `user_role` ADD PRIMARY KEY( `user_id`, `role_id`);

ALTER TABLE `role`   MODIFY `role_id` bigint(20) NOT NULL AUTO_INCREMENT;

ALTER TABLE `user`   MODIFY `user_id` bigint(20) NOT NULL AUTO_INCREMENT;

ALTER TABLE `user_role`   ADD CONSTRAINT `FK_user_id` FOREIGN KEY (`user_id`)
REFERENCES `user` (`user_id`),
  ADD CONSTRAINT `FK_role_id` FOREIGN KEY (`role_id`) REFERENCES `role`
(`role_id`);
```

2. Ajouter dans la base de données les utilisateurs avec leurs rôles, pour le mot de passe encodé en BCrypt, copiez le à partir de la console,

3. Dans la classe principale `ProduitsApplication, m`odifier méthode run comme suit :

```
@Autowired
PasswordEncoder passwordEncoder;

public static void main(String[] args) {
        SpringApplication.run(ProduitsApplication.class, args);
}

@Override
public void run(String... args) throws Exception {


        System.out.println("Password Encoded BCRYPT :******************* ");
         System.out.println(passwordEncoder.encode("123"));
/*produitService.saveProduit(new Produit("PC Dell", 2600.0, new Date()));
produitService.saveProduit(new Produit("PC Asus", 2800.0, new Date()));
produitService.saveProduit(new Produit("Imp Epson", 900.0, new Date()));
*/

}
```

4. Pour interroger les users à partir de la BD en utilisant JDBC ajouter la méthode suivante à la classe SecurityConfig :

```
@Bean
public UserDetailsService userDetailsService(DataSource dataSource) {
        JdbcUserDetailsManager jdbcUserDetailsManager =new
        JdbcUserDetailsManager(dataSource);

        jdbcUserDetailsManager.setUsersByUsernameQuery("select username ,
        password,   enabled from user where username =?");
        jdbcUserDetailsManager.setAuthoritiesByUsernameQuery("SELECT
        u.username, r.role as authority   " +
        "FROM user_role ur, user u , role r " +
"WHERE u.user_id = ur.user_id AND ur.role_id = r.role_id AND u.username = ?");

        return jdbcUserDetailsManager;
    }
```

Personnaliser l'interface Login et Logout

5. Modifier la méthode `defaultSecurityFilterChain` comme suit :

```
.requestMatchers("/login","/webjars/**").permitAll()
.anyRequest().authenticated())

                    .formLogin((formLogin) ->   formLogin
                                    .loginPage("/login")
                                    .defaultSuccessUrl("/") )

                    .httpBasic(Customizer.withDefaults())
```

6. Ajouter les méthodes *login()* et logout() au contrôleur SecurityController :

```
@GetMapping("/login")
 public String login()
 {
        return "login";
 }

@GetMapping("/logout")
 public String logout(HttpServletRequest request) throws ServletException
 {
        request.logout();
        return "redirect:/login";
 }
```

7. Créer la page login.html :

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
 xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
<head>
<meta charset="utf-8">
<title>Connexion</title>
</head>
<body>
   <div th:replace="template"></div>
        <div class="container mt-5">
          <div class="col-md-6 offset-md-3">
           <div class="card">
               <div class="card-header">Connexion</div>
                      <div class="card-body">
                             <form th:action="@{login}" method="post">
               <div class="form-group">
                      <label class="control-label">Utilisateur :</label>
                      <input       type="text" name="username">
               </div>
               <div class="form-group">
               <label class="control-label">Mot de passe :</label>
               <input       type="password" name="password">
               </div>
        <button class="btn btn-primary">Connexion</button>
                                    </form>
```

```
                    </div>
                </div>
            </div>
    </div>
</body>
</html>
```

L'authentification en utilisant JPA/Mysql

8. Créer dans le package entities l'entité User :

```
package com.nadhem.produits.entities;

import java.util.List;
import jakarta.persistence.CascadeType;
import jakarta.persistence.Entity;
import jakarta.persistence.Column;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.JoinTable;
import jakarta.persistence.ManyToMany;
import lombok.Data;
import lombok.AllArgsConstructor;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
@Entity
public class User {
 @Id
 @GeneratedValue (strategy=GenerationType.IDENTITY)
 private Long user_id;

 @Column(unique=true)
 private String username;
 private String password;
 private Boolean enabled;

 @ManyToMany(cascade=CascadeType.ALL, fetch = FetchType.EAGER)
 @JoinTable(name="user_role",joinColumns = @JoinColumn(name="user_id") ,
                inverseJoinColumns = @JoinColumn(name="role_id"))
 private List<Role> roles;
}
```

9. Créer dans le package entities l'entité Role :

```
package com.nadhem.produits.entities;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
```

```
import lombok.Data;
import lombok.AllArgsConstructor;
import lombok.NoArgsConstructor;


@Data
@AllArgsConstructor
@NoArgsConstructor
@Entity
public class Role {
 @Id
 @GeneratedValue (strategy=GenerationType.IDENTITY)
 private Long role_id;
 private String role;
}
```

10. Créer dans le package repos, l'interface UserRepository

```
package com.nadhem.produits.repos;

import org.springframework.data.jpa.repository.JpaRepository;
import com.nadhem.produits.entities.User;

public interface UserRepository extends JpaRepository<User, Long> {
 User findByUsername (String username);
}
```

11. Créer dans le package repos, l'interface RoleRepository

```
package com.nadhem.produits.repos;

import org.springframework.data.jpa.repository.JpaRepository;
import com.nadhem.produits.entities.Role;


public interface RoleRepository extends JpaRepository<Role, Long> {

 Role findByRole(String role);

}
```

Créer la couche service

12. Créer l'interface UserService

```
package com.nadhem.produits.service;

import com.nadhem.produits.entities.Role;
import com.nadhem.produits.entities.User;

public interface UserService {

 void deleteAllusers();
 void deleteAllRoles();
 User saveUser(User user);

 User findUserByUsername (String username);
 Role addRole(Role role);
```

```
        User addRoleToUser(String username, String rolename);

}
```

13. Créer l'implémentation UserServiceImpl

```
package com.nadhem.produits.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.nadhem.produits.entities.Role;
import com.nadhem.produits.entities.User;
import com.nadhem.produits.repos.RoleRepository;
import com.nadhem.produits.repos.UserRepository;

@Transactional
@Service
public class UserServiceImpl  implements UserService{

        @Autowired
        UserRepository userRep;

        @Autowired
        RoleRepository roleRep;


        @Autowired
        BCryptPasswordEncoder bCryptPasswordEncoder;

        @Override
        public User saveUser(User user) {

                user.setPassword(bCryptPasswordEncoder.encode(user.getPassword()));
                return userRep.save(user);
        }

        @Override
        public User addRoleToUser(String username, String rolename) {
                User usr = userRep.findByUsername(username);
                Role r = roleRep.findByRole(rolename);

                usr.getRoles().add(r);
                return usr;
        }

        @Override
        public void deleteAllusers() {
                userRep.deleteAll();

        }

        @Override
        public void deleteAllRoles() {
                roleRep.deleteAll();
```

```
    }

    @Override
    public Role addRole(Role role) {
          return roleRep.save(role);
    }

    @Override
    public User findUserByUsername(String username) {
          return userRep.findByUsername(username);
    }

}
```

14. On se propose, dans la suite de créer des utilisateurs en démarrant notre application, pour ce faire supprimer les tables user_role, rôle et user. Elles seront recréés au prochain redémarrage de l'application.

15. Modifier la classe *ProduitsApplication* pour ajouter les rôles et les utilisateurs au démarrage de l'application

```
@PostConstruct
  void init_users() {
        //ajouter les rôles
        userService.addRole(new Role(null,"ADMIN"));
        userService.addRole(new Role(null,"AGENT"));
        userService.addRole(new Role(null,"USER"));

        //ajouter les users
        userService.saveUser(new User(null,"admin","123",true,null));
        userService.saveUser(new User(null,"nadhem","123",true,null));
        userService.saveUser(new User(null,"user1","123",true,null));

        //ajouter les rôles aux users
        userService.addRoleToUser("admin", "ADMIN");

        userService.addRoleToUser("nadhem", "USER");
        userService.addRoleToUser("nadhem", "AGENT");

        userService.addRoleToUser("user1", "USER");
  }
```

Remarque :

Une fois les utilisateurs et leurs rôles sont enregistrés dans la base de données **commentez** la méthode init_users()

16. Dans la classe SecurityConfig, commentez la bean *userDetailsService* :
17. Créer la classe MyUserDetailsService qui implémente UserDetailsService:

```
package com.nadhem.produits.sercurity;

import java.util.ArrayList;
```

```java
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;
import com.nadhem.produits.entities.User;
import com.nadhem.produits.service.UserService;


@Service
public class MyUserDetailsService implements UserDetailsService {
   @Autowired
   UserService userService;

@Override
public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
   User user = userService.findUserByUsername(username);

if (user==null) throw new
        UsernameNotFoundException("Utilisateur introuvable !");

   List<GrantedAuthority> auths = new ArrayList<>();

    user.getRoles().forEach(role -> {
          GrantedAuthority auhority = new
SimpleGrantedAuthority(role.getRole());
          auths.add(auhority);
    });

   return new org.springframework.security.core.
                userdetails.User(user.getUsername(),user.getPassword(),auths);
  }
}
```

18. Testez votre travail